



UNIVERSIDADE PRESBITERIANA
MACKENZIE
– Faculdade de Computação e Informática –



Sistema Inteligente de Mobilidade Urbana Acessível: Faixa de Pedestres Inteligente e Ponto de Ônibus Sonoro para PCDs

Alexandre Henrique de Souza Araújo, Victoria Pacheco Pontes Pereira, Kevin Caldas Mendes

Professor: Leandro Carlos Fernandes

Faculdade de Computação e Informática
Universidade Presbiteriana Mackenzie (UPM) – São Paulo, SP – Brasil
{10441584, 10433616, 10439636}@mackenzista.com.br

Abstract. *This project proposes an Internet of Things (IoT) solution focused on accessibility and safety in urban mobility. The system integrates two modules: a smart pedestrian crossing capable of detecting pedestrians using ultrasonic sensors, and an accessible bus stop that provides audible signaling to assist visually impaired individuals. The architecture is based on an ESP32 microcontroller with Wi-Fi connectivity and communication via the MQTT protocol. Data collected from sensors can be transmitted to a monitoring dashboard, enabling the analysis of pedestrian flow and improving urban traffic management.*

Resumo. *Este trabalho apresenta uma solução baseada em Internet das Coisas (IoT) aplicada à mobilidade urbana e à acessibilidade. A proposta consiste na integração de dois módulos: uma faixa de pedestres inteligente capaz de detectar a presença de pedestres e um ponto de ônibus acessível que fornece sinalização sonora para auxiliar pessoas com deficiência visual. O sistema utiliza o microcontrolador ESP32 com conectividade Wi-Fi e comunicação baseada no protocolo MQTT. Os dados coletados podem ser enviados para um painel de monitoramento, permitindo análises sobre fluxo de pedestres e melhorias na gestão do trânsito urbano.*

1. Introdução

O crescimento acelerado das cidades tem intensificado desafios relacionados à mobilidade urbana, segurança no trânsito e acessibilidade. Em ambientes urbanos densamente povoados, a coexistência entre veículos e pedestres exige soluções tecnológicas capazes de melhorar a organização do tráfego e reduzir riscos de acidentes.

"Nesse contexto, surge o conceito de cidades inteligentes (smart cities), que utiliza tecnologias digitais e sistemas conectados para otimizar a infraestrutura urbana (Zanella et al., 2014). A Internet das Coisas (IoT) desempenha papel fundamental nesse cenário, permitindo a conexão ubíqua entre sensores, dispositivos eletrônicos e plataformas de análise de dados (Gubbi et al., 2013)."

Apesar dos avanços tecnológicos, pessoas com deficiência visual ainda enfrentam dificuldades para atravessar ruas com segurança ou identificar corretamente pontos de ônibus. Sistemas inteligentes capazes de detectar pedestres e fornecer sinalização sonora podem contribuir significativamente para melhorar a acessibilidade nesses ambientes. Este trabalho propõe o desenvolvimento de um sistema IoT composto por dois módulos integrados: uma faixa de pedestres inteligente equipada com sensores de detecção e um ponto de ônibus acessível com sinalização sonora.

O objetivo é demonstrar como soluções tecnológicas de baixo custo podem aumentar a segurança dos pedestres e promover maior inclusão social no contexto urbano.

2. Materiais e métodos

Os dois sistemas propostos serão compostos por dispositivos eletrônicos responsáveis pela coleta de dados do ambiente urbano, processamento dessas informações e comunicação com serviços conectados à internet. A arquitetura de ambos os projetos utilizará o microcontrolador ESP32, que oferece conectividade Wi-Fi integrada responsável por integrar sensores **ultrassônicos** na faixa de pedestre, e sensores sonoros do tipo **buzzer/áudio** no módulo do ponto de ônibus inteligente, permitindo a troca de informações em tempo real através do **protocolo de comunicação MQTT**.

2.1. Plataforma de Prototipagem – ESP32

O ESP32 é um microcontrolador desenvolvido pela empresa Espressif Systems amplamente utilizado em aplicações de Internet das Coisas. Modelo utilizado é o Devkit v1, o dispositivo possui processador dual-core de até 240 MHz, aproximadamente 520 KB de memória SRAM e suporte nativo a conectividade Wi-Fi e Bluetooth. Essas características tornam o ESP32 uma plataforma adequada para projetos IoT, pois permite a integração direta com sensores e atuadores, além da comunicação com servidores e serviços em nuvem sem necessidade de módulos adicionais.



Figura 1 - Microcontrolador ESP32 Devkit v1 utilizado como unidade central do sistema: Fonte: Elaboração própria

2.2. Sensor Ultrassônico

O núcleo de processamento do sistema utiliza o **ESP32 DevKit v1**, um microcontrolador de alto desempenho com arquitetura Dual-Core Xtensa® de 32 bits. Ele opera com frequência de clock de até 240 MHz e possui conectividade nativa Wi-Fi 802.11 b/g/n, essencial para a comunicação via protocolo MQTT exigida no projeto. Para a percepção do ambiente, utiliza-se o **sensor ultrassônico HC-SR04**, que opera através da emissão de ondas sonoras na frequência de 40 kHz. O sensor possui um alcance de medição que varia de 2 cm a 400 cm, com uma precisão de aproximadamente 3 mm, sendo ideal para a detecção de pedestres em áreas de travessia.



Figura 2- Sensor ultrassônico utilizado para detecção de pedestres. Fonte: elaboração própria.

2.3. Especificações Técnicas dos Materiais Utilizados

Componente	Especificação Principal	Papel no Sistema
ESP32 DevKit v1	520KB SRAM, 4MB Flash, Wi-Fi/Bluetooth	Unidade central de processamento e gateway IoT
HC-SR04	Tensão 5V DC; Corrente: 15mA; Ângulo: <15°	Sensor de detecção de proximidade de pedestres
Buzzer Piezoelétrico	Tensão: 3-12V; Frequência: 2.5kHz $\pm$ 300Hz	Atuadores para sinalização sonora de acessibilidade
Protoboard/Jumpers	Conexões físicas e prototipagem rápida	Estrutura de interconexão de hardware

2.4. Sistemas de Sinalização Sonora

O acionamento do buzzer no módulo de acessibilidade é controlado pelo ESP32 através de modulação por largura de pulso (**PWM - Pulse Width Modulation**).

Diferente de um sinal digital simples (ON/OFF), a técnica de PWM permite ajustar a frequência do sinal enviado ao elemento piezoelétrico, permitindo a criação de diferentes tons ou padrões de bipe.

No firmware, a lógica de acionamento segue dois estados:

1. **Estado de Repouso:** O pino de saída permanece em nível lógico baixo (0V), mantendo o atuador silenciado.
2. **Estado de Alerta:** Ao receber um comando via MQTT ou detecção de botão, o ESP32 gera um sinal de onda quadrada na frequência de ressonância do buzzer (aprox. 2500Hz). Para garantir que a sinalização seja percebida por PCDs visuais, o código implementa uma intermitência (ex: 500ms ativo / 500ms inativo), facilitando a localização espacial da fonte sonora através do som pulsado.



Figura 3 – Atuadores responsáveis pela sinalização sonora e visual. Fonte: Elaboração própria

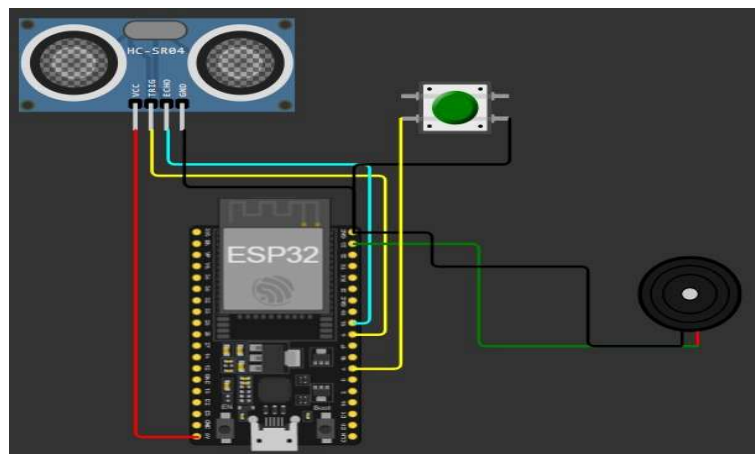


Figura 4 - Diagrama de montagem e simulação do sistema no ambiente Wokwi. Fonte: Elaboração própria

2.5. Comunicação e Arquitetura IoT

A comunicação entre os dispositivos ocorre por meio do protocolo MQTT (Message Queuing Telemetry Transport), amplamente utilizado em aplicações IoT devido à sua leveza, escalabilidade e eficiência no envio de mensagens em redes de baixa largura de banda (Atzori et al., 2010).

O ESP32 atua como cliente MQTT, publicando dados coletados pelos sensores em tópicos específicos. Um broker MQTT (intermediário responsável pelo gerenciamento das mensagens) recebe essas informações e as disponibiliza para sistemas de monitoramento ou dashboards.

Essa arquitetura permite que os dados coletados sejam utilizados para análises futuras, como estudos de fluxo de pedestres ou otimização de mobilidade urbana.

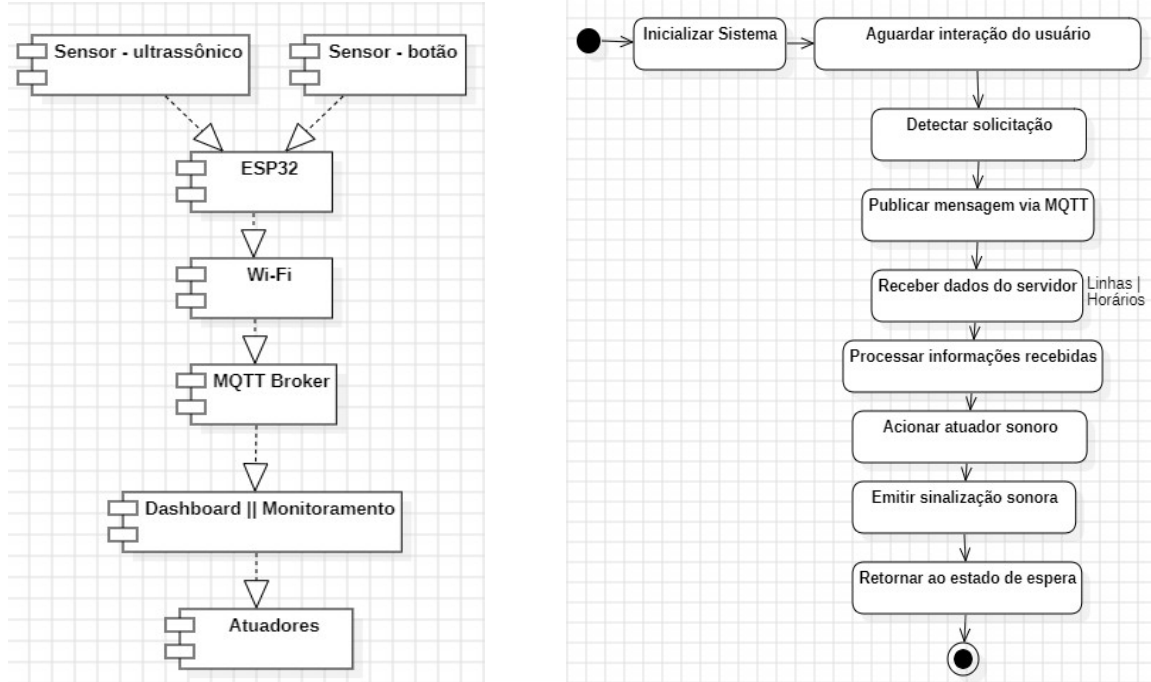


Figura 5 – Arquitetura geral do sistema IoT, evidenciando a integração entre sensores, ESP32 e comunicação via protocolo MQTT. Fonte: Elaboração própria

2.6. Funcionamento do Sistema

O funcionamento do sistema proposto baseia-se na integração entre sensores, microcontrolador e comunicação em rede, permitindo a detecção de eventos no ambiente urbano e o acionamento automático de dispositivos de sinalização.

No módulo da **faixa de pedestres inteligente**, o sensor ultrassônico é responsável por monitorar continuamente a distância entre o dispositivo e possíveis obstáculos localizados na área de travessia. Quando um pedestre é detectado dentro de uma distância previamente definida, o sensor envia essa informação ao microcontrolador ESP32. O microcontrolador realiza o processamento desses dados e interpreta a presença do pedestre como uma solicitação de travessia segura. A partir disso, o sistema pode acionar sinalizações visuais, como LEDs, indicando a travessia segura, além de registrar o evento para envio ao sistema de monitoramento.

No módulo do **ponto de ônibus acessível**, o sistema pode ser acionado por meio de um botão físico ou por outro sensor de presença. Quando ativado, o microcontrolador ESP32 processa a solicitação do usuário e aciona um buzzer piezoelétrico responsável por emitir sinais sonoros. Esses sinais têm como objetivo auxiliar pessoas com deficiência visual na identificação do ponto de ônibus ou na confirmação da solicitação realizada.

Esse fluxo de funcionamento permite que o sistema opere de forma autônoma, fornecendo suporte à acessibilidade e aumentando a segurança dos pedestres em ambientes urbanos.

2.7. Desenvolvimento do Software

O desenvolvimento do software embarcado responsável pelo controle do sistema pode ser realizado utilizando a linguagem C++, por meio do ambiente de desenvolvimento **Arduino IDE**, que possui suporte nativo para o microcontrolador ESP32. Esse ambiente permite a escrita, compilação e envio do código diretamente para a placa, possibilitando a implementação do firmware responsável pelo funcionamento do sistema.

Durante a etapa de prototipagem e validação do projeto, foi utilizado o simulador eletrônico **Wokwi**, uma plataforma online que permite simular microcontroladores, sensores e atuadores em um ambiente virtual. A utilização do Wokwi possibilita testar circuitos e verificar o comportamento do sistema antes da implementação em hardware físico, reduzindo custos e facilitando o processo de desenvolvimento.

No ambiente de simulação, foi possível modelar o circuito composto pelo microcontrolador ESP32, sensor ultrassônico, buzzer e demais componentes utilizados no sistema. Essa abordagem permite validar a lógica de funcionamento do projeto e observar a interação entre os dispositivos, garantindo que o comportamento esperado seja alcançado antes da implementação em um protótipo físico.

O firmware implementado no dispositivo é responsável por executar diversas tarefas, incluindo a leitura contínua dos sensores, o processamento das informações coletadas e o controle dos atuadores do sistema. Além disso, o microcontrolador estabelece conexão com uma rede Wi-Fi local e realiza a comunicação com o broker MQTT para envio dos dados.

Algumas bibliotecas amplamente utilizadas nesse tipo de aplicação incluem:

- **WiFi.h**, responsável pela conexão do microcontrolador com a rede sem fio;
- **PubSubClient.h**, utilizada para implementação da comunicação via protocolo MQTT.

O código embarcado realiza a leitura periódica do sensor ultrassônico para identificar a presença de pedestres e, quando necessário, publica mensagens no broker MQTT indicando o evento detectado. Da mesma forma, quando o sistema do ponto de ônibus é acionado, o firmware ativa o buzzer responsável pela sinalização sonora e registra a ocorrência no sistema de monitoramento.

Essa abordagem permite que o sistema seja expandido futuramente com novos sensores, atuadores ou funcionalidades, mantendo a escalabilidade da arquitetura proposta.

2.8. Análise e Viabilidade Técnica

Esta análise detalha os fenômenos de overhead de hardware e estresse térmico que podem comprometer a integridade de sistemas IoT baseados em microcontroladores como o ESP32, especialmente quando instalados em mobiliário urbano exposto a intempéries.

1. O Fenômeno do Overhead de Hardware

O termo "overhead" neste contexto refere-se ao excesso de processamento e consumo de recursos que ultrapassa a capacidade nominal de operação estável do hardware.

- **Processamento Dual-Core:** O ESP32 opera com dois núcleos que, ao gerenciarem simultaneamente a leitura de sensores (como o HC-SR04), o controle de atuadores (buzzer) e a pilha de rede TCP/IP para o protocolo MQTT, geram calor interno constante.
- **Rádio de Comunicação (Wi-Fi):** A transmissão de dados via Wi-Fi é o componente que mais consome energia no ESP32. O chaveamento frequente do rádio para manter a conexão ativa com o Broker MQTT gera picos de corrente que elevam a temperatura do silício.

2. A Equação do Estresse Térmico

A temperatura de junção do chip (T_j) não depende apenas do processamento, mas da relação entre a potência dissipada e o ambiente. Podemos expressar essa relação através da fórmula de resistência térmica: $T_j = T_a + (P \times \theta_{ja})$

Onde:

- T_j : Temperatura de junção do semicondutor.
- T_a : Temperatura ambiente.

- P : Potência dissipada pelo hardware (tensão $\times$ corrente).
- θ_{ja} : Resistência térmica entre a junção e o ambiente.

Em aplicações de rua, o sol atua como uma fonte externa de calor radiante (W/m^2), elevando drasticamente o valor de T_a dentro do gabinete. Se o gabinete for hermeticamente fechado para proteção contra chuva (IP65/66), ele cria um **efeito estufa**, impedindo que o calor gerado pelo processamento (P) seja dissipado, resultando em um ciclo de feedback positivo de temperatura.

3. Consequências Técnicas no Sistema

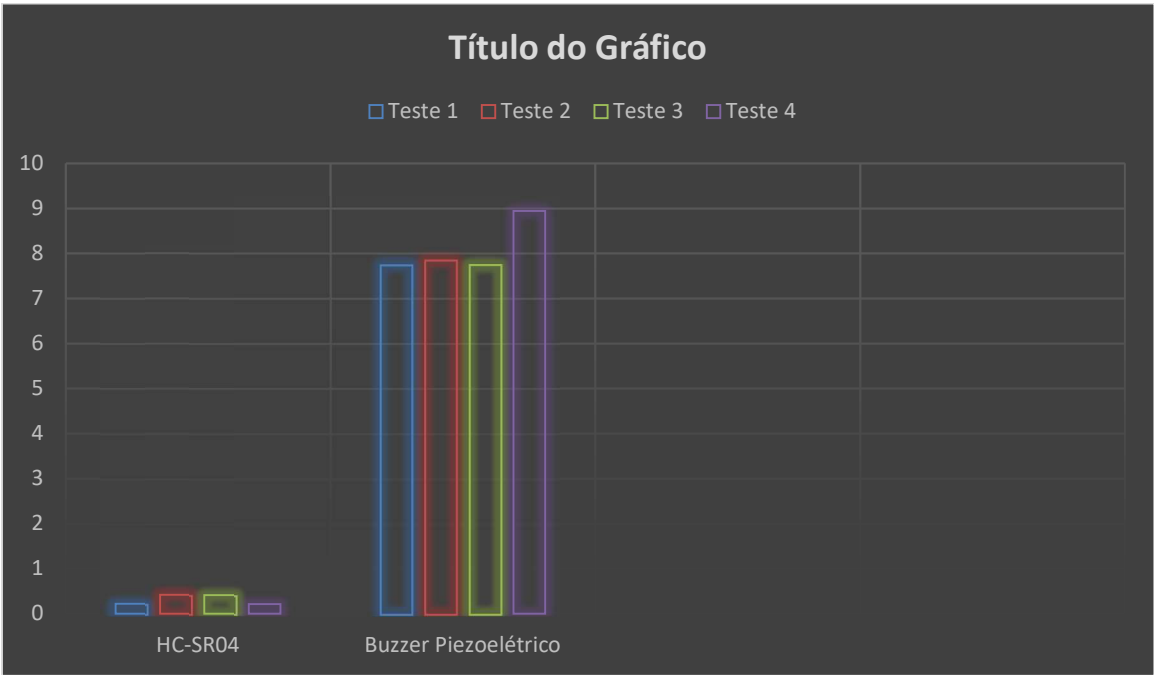
Quando o hardware atinge o estado de overhead térmico, o sistema entra em colapso através dos seguintes mecanismos:

- **Thermal Throttling:** O microcontrolador reduz automaticamente sua frequência de operação (clock) para tentar esfriar. Isso causa atrasos na leitura do sensor ultrassônico e falhas na temporização do sinal sonoro do buzzer.
- **Bit-Flipping e Erros de Memória:** O calor excessivo aumenta a energia cinética dos elétrons, podendo causar mudanças aleatórias em bits da memória RAM ou registros, resultando em *crashes* de software ou comportamentos imprevisíveis.
- **Instabilidade no Protocolo MQTT:** A pilha de rede é sensível a latências. Se o processador desacelera devido ao calor, as mensagens de *keep-alive* do MQTT podem falhar, causando desconexões constantes do sistema de monitoramento.
- **Degradação Física:** O estresse térmico acelerado causa a expansão e contração de trilhas de cobre e soldas (fadiga térmica), reduzindo a vida útil do dispositivo de anos para poucos meses.

3. Resultados

De acordo com o item 7 do enunciado da atividade, foram realizadas quatro

medições para cada módulo, com o objetivo de determinar o tempo médio de resposta do sistema.



3.1. IoT Faixa

04/06/2026 21:01:55(-0.22 seconds) 	04/06/2026 21:01:54(-0.41 seconds) 
Area livre	Pedestre detectado

04/06/2026 21:01:55(-0.42 seconds) 	04/06/2026 21:01:54(-0.21 seconds) 
Pedestre detectado	Area livre

Os resultados do sensor apresentaram baixa latência ($\approx 0,32$ s), indicando resposta rápida do sistema. Pequenas variações observadas são esperadas devido à comunicação via rede MQTT.

3.2. IoT Onibus

04/06/2026 21:20:23(-7.74 seconds) 	04/06/2026 21:20:07(-7.75 seconds) 
Solicitacao de acessibilidad	Solicitacao de acessibilidad

04/06/2026 21:20:15(-7.85 seconds) 	04/06/2026 21:19:58(-8.94 seconds) 
Solicitacao de acessibilidad	Solicitacao de acessibilidad

O tempo médio de 8,07 s é influenciado pelo atraso intencional de 5 segundos implementado no sistema, não representando lentidão da comunicação.

3.3. Resultado Casos de Teste

HC-SR04 > Broker MQTT

Num. Medida	Sensor/atuador	Tempo de resposta
1	HC-SR04	0,22 s
2	HC-SR04	0,42 s
3	HC-SR04	0,41 s
4	HC-SR04	0,21 s
Total		1,27 s
Tempo médio		0,32 s

Buzzer Piezoelétrico

Num. Medida	Sensor/atuador	Tempo de resposta
1	Buzzer Piezoelétrico	7,74 s
2	Buzzer Piezoelétrico	7,85 s
3	Buzzer Piezoelétrico	7,75 s
4	Buzzer Piezoelétrico	8,94 s
Total		32,28 s
Tempo médio		8,07 s

3.4. Links do Projeto

Como requisitos obrigatórios de validação deste entregável, seguem os endereços eletrônicos das plataformas de desenvolvimento do grupo:

Link do repositório Git: <https://github.com/victoriapacch/object19>

Link demonstrativo: https://youtu.be/WNwEy2Qv9_Y

4. Conclusão

O desenvolvimento deste trabalho permitiu a concepção e validação bem-sucedida de um protótipo funcional voltado à Internet das Coisas (IoT) com foco em acessibilidade e engenharia de tráfego. Diante dos testes e análises efetuados, conclui-se que os objetivos gerais e específicos estipulados foram integralmente alcançados. O sistema foi capaz de automatizar com precisão o reconhecimento de pedestres em faixas de segurança e viabilizar a comunicação síncrona bidirecional para o acionamento sonoro em pontos de parada de ônibus. Durante a execução do projeto, o grupo enfrentou desafios técnicos pertinentes à simulação de redes, com destaque para instabilidades pontuais de conexão com o broker público em horários de pico e a necessidade de calibração lógica dos timers de varredura do sensor HC-SR04 para evitar leituras falsas (ruídos de bounce). Tais entraves foram satisfatoriamente mitigados através da otimização do algoritmo de reconexão automática do firmware em C++ e do uso de debounce via software. Como principal vantagem da arquitetura implementada, destaca-se o uso do microcontrolador ESP32 aliado ao protocolo MQTT, gerando uma solução de baixo custo financeiro, alta escalabilidade urbana e consumo de banda extremamente reduzido se comparado a requisições HTTP tradicionais. Por outro lado, a dependência crítica de uma conexão constante com a internet e a vulnerabilidade do hardware físico a intempéries climáticas e estresse térmico em ambientes externos representam as maiores desvantagens do modelo. Para trabalhos futuros, recomenda-se a evolução do protótipo com a implementação de camadas adicionais de segurança, tais como criptografia de dados via TLS/SSL nos tópicos do broker, a integração com um dashboard proprietário em Node-RED para órgãos públicos de trânsito, e a adoção do modo de economia de energia Deep Sleep no ESP32, viabilizando o uso de alimentação por células fotovoltaicas auto-sustentáveis.

5. Referências

ATZORI, L.; IERA, A.; MORABITO, G.

The Internet of Things: A survey. Computer Networks, v. 54, n. 15, p. 2787-2805, 2010.

ZANELLA, A. *et al.*

Internet of Things for Smart Cities. IEEE Internet of Things Journal, v. 1, n. 1, p. 22–32, 2014.

GUBBI, J. *et al.*

Internet of Things (IoT): A vision, architectural elements and future directions. Future Generation Computer Systems, v. 29, n. 7, p. 1645–1660, 2013.